

ONIONVISION

AI-Based Onion Quality Inspection & Automated Grading System

Project Master Plan / Product Requirements / Technical Blueprint

Team: THE DEBUGGERS

Target prototype deadline: 27 September 2026

Purpose: Single source of truth for architecture, datasets, machine learning, backend, frontend, testing, reports and SIH demonstration.

0. DOCUMENT STATUS & SOURCE BASIS

This document is the engineering/product blueprint for an independently implemented OnionVision prototype. A previously provided Cepa project is treated as a reference for understanding a similar computer-vision workflow, not as source code to disguise or submit as original work.

The exact dataset labels, annotation structure and capabilities must be verified from the downloaded datasets before model training. This document does not invent agricultural standards, defect labels, accuracy figures or production-grade claims.

Important: A/B/C/Reject values described here are prototype design concepts until an appropriate agricultural standard is verified and implemented.

1. EXECUTIVE SUMMARY

OnionVision is a computer-vision-based inspection and grading prototype intended to analyze photographs of harvested onions, identify individual onions, estimate image-derived measurements, identify supported visible quality/defect classes, calculate an explainable prototype quality score, and produce a prototype grade together with batch analytics and an inspection report.

The system separates model training from production inference. Datasets are audited first; models are trained in the ML workspace; trained artifacts are versioned; FastAPI exposes inference through a controlled API; and the React frontend consumes structured results.

2. PROJECT OBJECTIVES

Create a functioning onion image inspection workflow rather than a static UI mockup.

Detect or segment individual onions in a submitted image where the selected model supports that task.

Calculate image-derived area, contour and dimensions, and estimated physical size only when a valid calibration reference is available.

Classify only quality/defect categories actually supported by the selected training data.

Produce an explainable prototype grade based on measurable features and documented prototype rules.

Provide batch statistics such as total onions, grade distribution, average estimated size and supported visible defect rate.

Store inspection history and allow previous results to be revisited.

Generate a professional inspection report with a traceable inspection identifier.

Provide a farmer/operator-friendly workflow.

Clearly separate prototype capability from claims requiring agricultural validation.

3. PROBLEM TO BE SOLVED

Manual visual inspection of onions can be repetitive and dependent on the inspector, lighting, viewing angle and consistency of human judgement. OnionVision explores whether computer vision can make a photographed sample easier to inspect by providing consistent image-based measurements, visible-quality indicators and a repeatable prototype scoring workflow.

The system is not intended to replace certified agricultural grading, laboratory analysis or human inspection where internal defects, chemical properties, moisture, firmness, disease diagnosis or legally defined standards require additional evidence.

4. CORE USER WORKFLOW

1. Operator opens OnionVision → 2. Starts New Inspection → 3. Uploads/captures image → 4. Image validation → 5. Segmentation/detection → 6. Individual onion extraction → 7. Measurement and quality analysis → 8. Explainable prototype grade → 9. Persistence → 10. Batch analytics → 11. Report.

5. FINAL DEMO FLOW

Dashboard → New Inspection → Upload real onion image → show analysis progress → display segmentation overlays → inspect an individual onion → show size/quality/confidence/reason → display batch distribution → open history → generate

report → explain limitations.

6. SYSTEM ARCHITECTURE

User → React/TypeScript frontend → FastAPI REST API → Inspection Service → ML Inference Pipeline → Segmentation Model + Quality Model → Measurement Service → Prototype Grading Engine → SQLite → Results/Analytics/Report → Frontend.

Training is intentionally separate from inference. Training scripts consume datasets and produce model artifacts. The backend loads model artifacts only for prediction. This prevents model training inside API requests and keeps the system replaceable and testable.

7. TECHNOLOGY STACK

Layer	Technology	Purpose
Frontend	React + TypeScript + Vite	Responsive inspection dashboard and workflow
Styling	Tailwind CSS	Consistent UI implementation
Charts	Recharts	Inspection and grade analytics
Backend	Python + FastAPI	REST API and inference orchestration
Validation	Pydantic	Typed request/response validation
Database	SQLite + SQLAlchemy	MVP persistence/history
Vision	OpenCV / Pillow / NumPy	Image processing and measurements
ML	Framework selected after dataset audit	Segmentation and quality models
Reports	Python PDF/reporting layer	Inspection reports
Testing	pytest + API tests	Backend correctness/regression

8. REPOSITORY STRUCTURE

```
ONIONVISION/  
■■■ backend/app/api/ - HTTP routes  
■■■ backend/app/schemas/ - API schemas  
■■■ backend/app/services/ - business logic  
■■■ backend/app/ml/ - inference adapters  
■■■ backend/app/db/ - database layer  
■■■ backend/app/core/ - configuration  
■■■ ml/preprocessing/ - dataset preparation  
■■■ ml/training/ - training scripts  
■■■ ml/evaluation/ - evaluation  
■■■ ml/models/ - trained artifacts  
■■■ data/raw/ - untouched datasets  
■■■ data/processed/ - derived data  
■■■ data/manifests/ - provenance  
■■■ docs/ - engineering documentation  
■■■ reports/ - generated reports
```

Raw datasets remain untouched. Processing produces separate derived data.

9. DATASET STRATEGY

Three onion datasets have already been downloaded. They must be audited before training because their labels, image conditions, annotations and licensing may differ. They should not be mixed blindly.

Dataset	Intended role	Next action
Bad Onion	Visible onion quality/defect learning	Audit exact classes, counts, image conditions and license
Red & White Onion (~16,300)	Healthy/unhealthy and bulb-related classification where labels support bulb/leaf composition and usable subset	Audit COCO categories and annotation quality
Onion Segmentation (~4,849)	Individual onion segmentation	Audit COCO categories and annotation quality
VegQual	Potential fresh/defective support	Download later only after verifying a valid source

10. DATASET AUDIT REQUIREMENTS

- Count images and annotation files.
- Read class/category definitions from source annotations instead of guessing.
- Inspect dimensions, channels, corrupted files and unusual samples.
- Determine existing train/validation/test splits.
- Create a reproducible split if needed.
- Check duplicate or near-duplicate images across splits.
- Record source, version/date, license, citation and intended use.
- Keep raw data untouched; create processed copies.
- Never use labels that are not explicitly present.
- Never claim a defect category unless represented in training/evaluation data.

11. MODELING PLAN

Component	Input	Output	Purpose
Segmentation	Full onion image	Individual onion masks/instances	Locate separate onions
Quality model	Individual onion crop	Supported quality/defect class + confidence	Visual quality analysis
Measurement	Mask + valid calibration/reference	Area, dimensions, estimated mm if valid	Quantitative features
Grading engine	Measurements + quality output	Score + prototype grade + reasons	Explainable decision
Batch analytics	Individual results	Counts/rates/distributions	Operator summary

12. SEGMENTATION PIPELINE

Image → preprocessing → segmentation model → instance masks → mask cleanup → contour checks → individual onion crops → measurements.

The segmentation dataset was downloaded in COCO Segmentation format. Exact category names must be read from the COCO JSON before training.

13. MEASUREMENT PIPELINE

A mask provides pixel area, contour and bounding-box dimensions. Physical millimetres require a known reference or calibrated imaging setup. The application must distinguish pixel measurements from estimated physical measurements.

Illustrative calibration: $\text{scale} = \text{reference_pixels} / \text{reference_mm}$; estimated onion diameter = $\text{onion_pixels} / \text{scale}$. This is an engineering example, not a validated agricultural measurement standard.

14. QUALITY / DEFECT PIPELINE

Each detected onion can be cropped and passed through the quality model. The output contains a class and confidence. Low confidence must remain visible rather than being converted into a definitive claim.

If a dataset supports only healthy/unhealthy, the UI must use those labels or their documented equivalents. It must not silently rename them as bruising, rot, sprouting or other unsupported categories.

15. PROTOTYPE GRADING ENGINE

The grading engine is deterministic and explainable. It combines supported visual quality information with measurement features. Thresholds remain configurable and documented. Unless a recognized standard is verified, A/B/C/Reject are prototype grades, not official certification.

Example: Grade B; quality score 82.4/100; reasons: acceptable estimated size; supported visible quality issue detected; confidence above configured review threshold.

16. CONFIDENCE & MANUAL REVIEW

Low-confidence predictions should enter a review state. The operator can see the image, prediction and confidence and can manually review/correct. Store manual corrections separately from original model predictions so evaluation data is not silently overwritten.

17. BACKEND API CONTRACT

Method	Endpoint	Purpose
GET	/api/health	Backend health
POST	/api/inspections	Upload image/create inspection
GET	/api/inspections	List inspection history
GET	/api/inspections/{id}	Retrieve complete inspection
GET	/api/inspections/{id}/results	Retrieve onion results
GET	/api/inspections/{id}/report	Generate/retrieve report

18. API RESPONSE MODEL

```
{
  "inspection_id": "INS-20260925-001",
  "status": "completed",
  "total_onions": 12,
  "average_size_mm": 58.4,
  "quality_score": 84.2,
  "defect_rate": 12.5,
  "grade_distribution": {"A": 8, "B": 3, "C": 1, "Reject": 0},
  "onions": [{"id": 1, "size_mm": 61.2, "quality_class": "supported_label", "grade": "A", "confidence": 0.94}]}

```

19. DATABASE DESIGN

Table	Key fields	Purpose
inspections	id, inspection_id, image_path, status, timestamps, totals	One record per inspection
onion_results	id, inspection_id, onion_number, size, class, grade, confidence	Individual results
reports	id, inspection_id, file_path, created_at	Report references

20. FRONTEND PLAN

Frontend is intentionally postponed until backend/model outputs are known. It will be built directly against the verified API contract.

Screen	Main content
Dashboard	Inspection count, average quality, grade distribution, defect rate, recent inspections
New Inspection	Upload/camera, preview, validation, Analyze
Analysis	Validation, segmentation, measurement, quality progress
Results	Detected count, quality score, grades, overlays
Individual Onion	Size, supported class, confidence, grade, explanation
History	Previous inspections and filters
Report	Professional report with inspection ID and verification reference

21. FRONTEND UX PRINCIPLES

- Make New Inspection the obvious primary action.
- Show the analyzed image and trace AI results to it.

- Use clear states: Uploading, Processing, Completed, Failed, Review Required.
- Never display an unexplained grade.
- Show confidence separately from quality score.
- Keep the workflow responsive on laptop/tablet/phone.
- Use useful error messages for invalid images and model failures.

22. REPORT DESIGN

Report should contain: branding, inspection ID, date/time, sample reference, total inspected, average estimated size where available, grade distribution, supported quality classes, defect rate where supported, quality score, model/version identifiers, review status, limitations and verification reference.

Do not state that OnionVision is an officially certified grading authority unless certification has actually been obtained.

23. MODEL EVALUATION PLAN

Task	Metrics
Segmentation	IoU / mAP or task-appropriate segmentation metrics; class-wise performance
Classification	Accuracy, precision, recall, F1, confusion matrix
Confidence	Confidence distribution and error analysis
Measurement	MAE/RMSE against valid ground truth
End-to-end	Completion rate, latency, repeatability and failure handling

24. TRAIN / VALIDATION / TEST RULES

- Never evaluate on training images.
- Use reproducible splits.
- Keep images from the same physical sample in one split when identifiable.
- Record random seeds and preprocessing versions.
- Keep final test set untouched until model selection is complete.
- Report class imbalance and confusion matrices, not only accuracy.
- Review false positives and false negatives manually.

25. PERFORMANCE TARGETS

Prioritize a stable end-to-end demonstration over an arbitrary accuracy target. Only publish actual evaluation numbers produced by the trained model. Latency should be measured after model selection.

26. SECURITY & DATA HANDLING

- Validate upload type and size.
- Generate server-side unique file names.
- Prevent path traversal.
- Do not expose raw storage unnecessarily.
- Keep secrets in environment variables.
- Do not commit .env.
- Do not log sensitive image data unnecessarily.
- Use stronger authentication/storage controls for any production deployment.

27. TESTING STRATEGY

Area	Tests
Backend	Health, upload validation, creation, retrieval, 404 handling
Database	Initialization, relationships, persistence
ML	Model load, inference schema, controlled missing-model state
Grading	Normal/boundary cases, missing measurements, low confidence
Frontend	Upload, loading, results, history, report, errors
Integration	Image → API → model → grade → DB → response
Demo	Real images, different onion counts, poor images, review path

28. FAILURE / EDGE CASES

- No onion detected.
- One onion detected.
- Overlapping onions.
- Dark, overexposed or blurred image.
- Unsupported or oversized file.
- Model unavailable.
- Low-confidence prediction.
- Missing calibration reference.
- Invalid/tiny segmentation mask.
- Database or report generation failure.
- User refreshes during processing.

29. PHASED IMPLEMENTATION PLAN

Phase	Work	Deliverable
01	Backend foundation	FastAPI + SQLite + API contract + tests
02	Dataset audit	Manifest + classes + counts + split plan
03	ML training	Models + evaluation + versioned artifacts
04	Inference pipeline	Image → segmentation → measurement → quality → grade
05	Backend integration	Real inference endpoint + persistence
06	Frontend	Dashboard + inspection + results + history
07	Reports/analytics	PDF + charts + verification
08	Testing/polish	Bug fixes + demo + final docs

30. 48-HOUR EXECUTION PLAN

Day 1: backend foundation, dataset audit, preprocessing, model selection, segmentation training and inference skeleton.

Day 2: finish model evaluation, FastAPI integration, frontend inspection/results/dashboard, history, report and testing.

Safety buffer: reserve several hours for model/runtime issues, dependency conflicts and demo stabilization.

31. WHAT WE WILL NOT BUILD IN THE EMERGENCY MVP

- No microservices, Kubernetes or unnecessary cloud infrastructure.
- No heavy authentication unless required.

- No mobile app before the core web prototype is stable.
- No fake AI predictions.
- No unsupported defect classes.
- No official certification claims.
- No automatic Git push.
- No unnecessary large-scale experiments after the end-to-end demo works.

32. SIH DEMONSTRATION TALKING POINTS

Problem: manual visual inspection can be repetitive and inconsistent.

Solution: OnionVision turns a photographed onion sample into an explainable computer-vision inspection report.

Technology: segmentation, image measurement, quality classification, deterministic grading, FastAPI, React and SQLite.

Innovation: traceable per-onion analysis, explainable grading, batch analytics and a field-oriented inspection workflow.

Limit: prototype analyzes visible image information and is not a substitute for certified grading or internal-quality testing.

33. DEFINITION OF DONE

- Backend starts cleanly.
- Health endpoint works.
- Image upload and persistence work.
- Actual trained model artifacts load for inference.
- Segmentation can be visualized.
- Quality predictions use only supported labels.
- Prototype grade is explainable.
- History and report work.
- Frontend handles loading/error/success states.
- Tests pass.
- Real sample works end-to-end.
- Dataset provenance and documentation are complete.
- No unsupported claims appear in UI/PPT.

34. FINAL PROJECT OUTPUTS

Output	Expected artifact
Source	ONIONVISION repository
Backend	FastAPI application
Database	SQLite MVP
Segmentation model	Versioned artifact + evaluation
Quality model	Versioned artifact + evaluation, if supported
Dataset manifest	Source, version, license, classes, preprocessing, citation
Frontend	Responsive React/TypeScript application
Report	PDF inspection report
Documentation	README, architecture, API, ML docs
Demo	End-to-end inspection with real images
PPT	SIH-compliant six-slide presentation

35. MASTER ENGINEERING RULES

- Build the smallest system that honestly demonstrates the PS.
- Use real datasets and preserve provenance.
- Never invent labels, standards, accuracy or AI capabilities.
- Frontend must consume fields the backend actually produces.
- Backend must consume outputs the model actually produces.
- Keep training reproducible and model artifacts versioned.
- Separate raw, processed and generated data.
- Test every phase before moving forward.
- Prefer explainability for grading.
- Use manual review for uncertain predictions.
- Keep architecture simple enough to finish under the deadline.
- Never confuse prototype grades with official agricultural certification.

36. IMMEDIATE NEXT ACTION

Start Phase 01: Backend + ML Foundation. After verification, perform Phase 02: Dataset Audit on the three downloaded datasets. Only then select and train exact segmentation and quality models.

This document is the project source of truth and should be updated when a major architectural or capability decision changes.